

Studio Stack

Agency Devs

Neema Owji

Zachary Tenn Yuk

CIS 4914 Senior Project

Mark Rottensteiner

mrottensteiner@ufl.edu

Final Presentation Link: <https://www.youtube.com/watch?v=N3Zj-hTi4BI>

Keywords

Equipment management, inventory management, full-stack application, production

Project Overview

Our project, Studio Stack, is an inventory management system designed specifically for The Agency at UF. The Agency is a marketing agency that allows students at UF to work with brands to solve real-world problems. An important part of The Agency is the production department, which handles all video and photo production. Because The Agency handles multiple clients at once, there can be multiple production teams simultaneously working on different shoots throughout the semester. This makes it hard to know if a piece of equipment will be available when it's needed, especially without a formalized way to manage all inventory.

Furthermore, the equipment used by the production team, including cameras, microphones, and lighting equipment, is very expensive, with the total cost of items being used for a shoot being in the thousands of dollars. This makes it even more important to have a way of tracking who has what piece of equipment and keeping a history of the chain of custody. If equipment becomes damaged or broken during a shoot, The Agency needs to know who had it last.

The main objectives of the inventory management system were to have an intuitive user interface that provided a low-friction way to reserve equipment, check items in and out, and provide The Agency administration with a high-level overview of how equipment is used. We wanted to have a centralized web application that could be used on either desktop or mobile

devices, so anyone could access it at the same time. We also needed to create a single source of truth as to who would be able to use a piece of equipment at any point in time.

The development team consisted of two individuals, Neema and Zach. Both developers worked on the entire stack and tasks were evenly assigned based on the amount of effort they took. Neema, in addition to feature development and bug fixes, also focused more on planning and testing. Zach, on the other hand, led most of the meetings and interfaced heavily with both the production team and the faculty advisor. This division of labor allowed both developers to be focused on their specific programming tasks while also keeping the project on track.

Deliverables

GitHub Code Repository: <https://github.com/The-Agency-at-UF/studiostack>

Firebase: Firestore Database storing information for bug reports, inventory, mail, notifications, reports, reservations, teams, and users. This Firebase project is hosted privately on the Agency's Developer Google Account.

Functional Specifications

The main function of Studio Stack is its reservation system. Users can make a reservation for a certain time and book equipment. These reservations are saved in the system and prevent others from double-booking equipment. Reservations can be created, edited, and deleted from the system. The reservations are viewable on a calendar page. Users can also create reports if equipment is damaged, and view statistics generated from analyzing submitted reports. Users, project teams, and equipment inventory are stored in a Firebase database and can be edited through admin dashboards. Bugs can also be reported by users via an in-site form.

Current Status

Previously, the app was not used by the Agency due to poor functionality and a buggy equipment barcode scanning system. Our improvements included overhauling the scanning system to use QR code scanning via device cameras, streamlining the UI with more integrated error notifications, and a bug reporting feature for app feedback. With our current system, we have received overwhelmingly positive feedback from our users. The management team is very happy with the admin features and dashboards, and production team members find the QR codes intuitive, making it easy to handle reservations. We expect the project to live on in The Agency and that the production team will continue to use the app, with other developers being recruited for occasional maintenance and feature requests.

Known Issues and Risks

- When submitted by a user, bug reports are currently stored in the Firestore database.

Future functionality could include notifications for developers and an Admin dashboard.

Technical Details

Studio Stack was developed as a web app using React with JavaScript for the frontend and Firebase for the backend, authentication, deployment, and database. We used a number of NPM packages in order to reduce our development time. These include fullcalendar, which let us make an effective calendar without having to write one from scratch; html5-qrcode, which handled both QR code generation and scanning, streamlining the checkout process; tailwindcss in order to easily style the web app; and recharts for creating easy-to-understand visuals for our reports.

Firebase was integral for providing a fully implemented solution before the project deadline. It offers a wide suite of solutions, and we integrated them extensively during development. We used the Firestore feature for a non-relational database with multiple collections. This stored everything that we needed, including reservations, production teams, the inventory, whitelisted users, and reports. Every database entry is given a randomized, unique ID and any information that needs to be stored is attached to it in a JSON format. The other Firebase solutions we used are authentication so that users can sign in with their UFL emails; the hosting feature so that our app is deployed every time there is a new commit to the main branch; and functions so that, when an item is overdue or broken.

The codebase is stored on GitHub so that we can take advantage of branch protections and GitHub actions for a CI/CD pipeline. The deployed version of Studio Stack is based on the main branch, which requires a pull request from another branch and an approval from a developer. To manage this, we utilized a feature branching schema so that new features are isolated from the main code until everything is tested and so we know which developer is working on a feature. We also implemented a CI/CD pipeline with GitHub Actions to streamline deployment and testing. Every time new code is merged into the main branch, the pipeline runs our testing suite and then deploys the app onto our Firebase instance.

For deployment, the GitHub Actions pipeline needs to have the following environment variables saved:

- VITE_FIREBASE_API_KEY
- VITE_FIREBASE_AUTH_DOMAIN
- VITE_FIREBASE_PROJECT_ID
- VITE_FIREBASE_STORAGE_BUCKET

- VITE_FIREBASE_MESSAGING_SENDER_ID
- VITE_FIREBASE_APP_ID
- VITE_FIREBASE_MEASUREMENT_ID

All of these variables can be retrieved from the Firebase console.

Testing Information

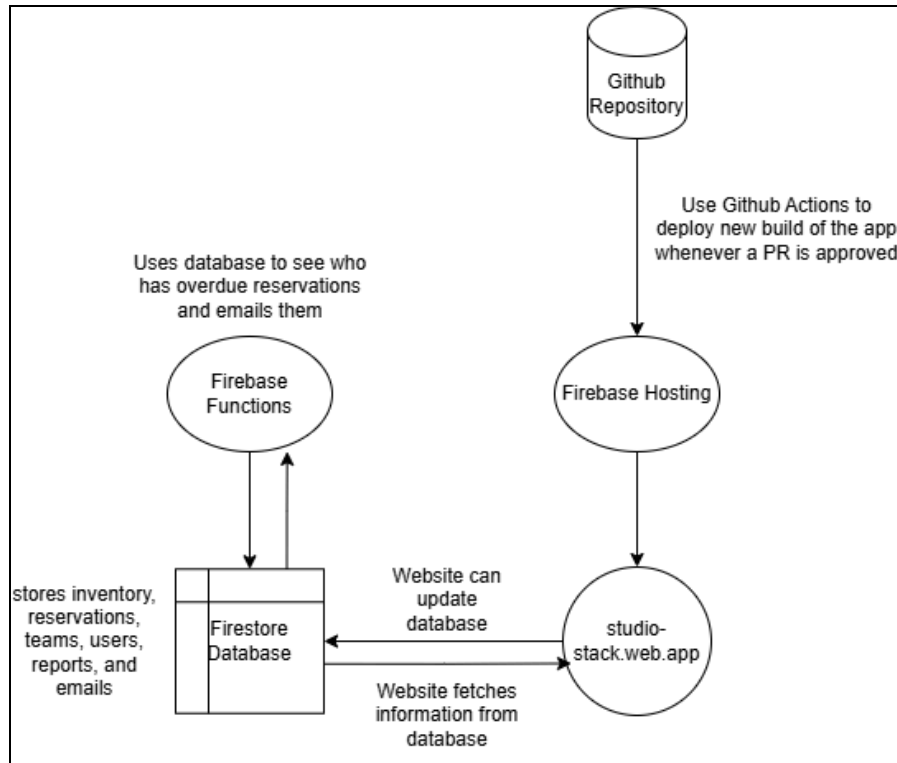
For testing, we created a full suite of unit and integration tests using the Vitest library. Every page has a full integration test that makes sure that a user flow works as expected with all the components used. For example, the Calendar page has a test that makes sure the correct reservation details are displayed when a reservation is clicked on. This also requires creating a mock API for the Firebase database so that we can use dummy data and test with that.

In addition, complex and important components are given individual unit tests so that their behavior remains predictable. The Header component, which is shown on every page in the app, has its own unit tests to make sure that things like admin features only showing on admin accounts always work. This way, whenever a change is made, the developer can run the test suite and know that there are no unintended consequences.

In total, there are 13 test files and 38 test cases. The full list of tests is provided in the appendix, along with a screenshot showing that they all pass.

User Documentation

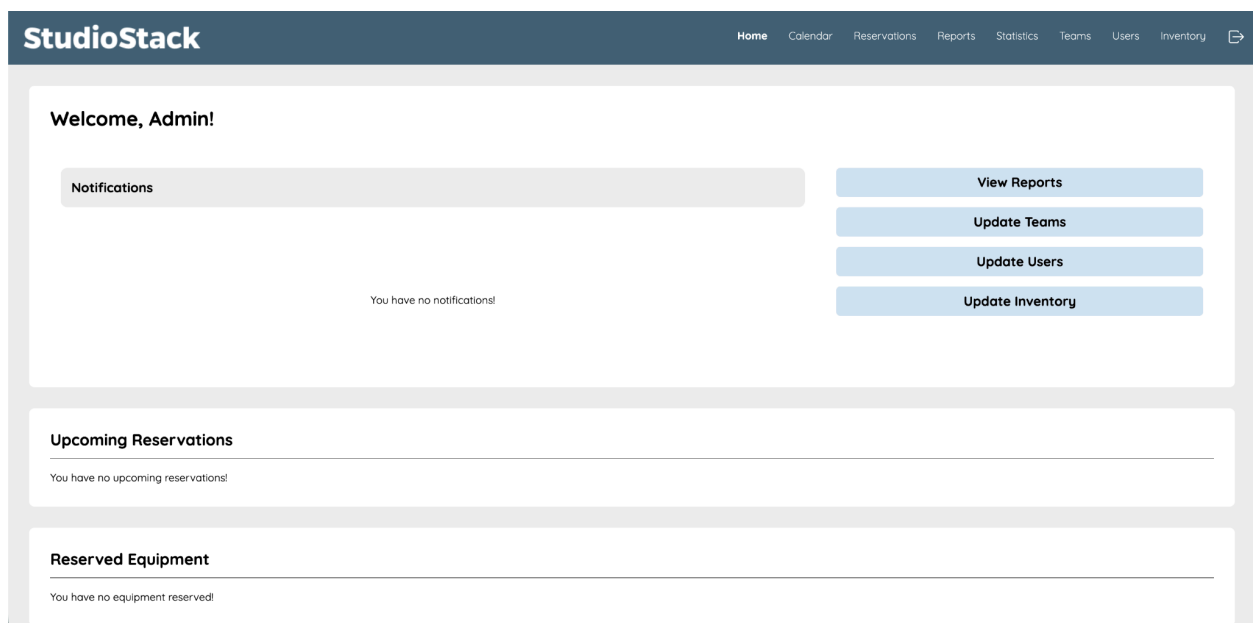
Software Architecture Diagram



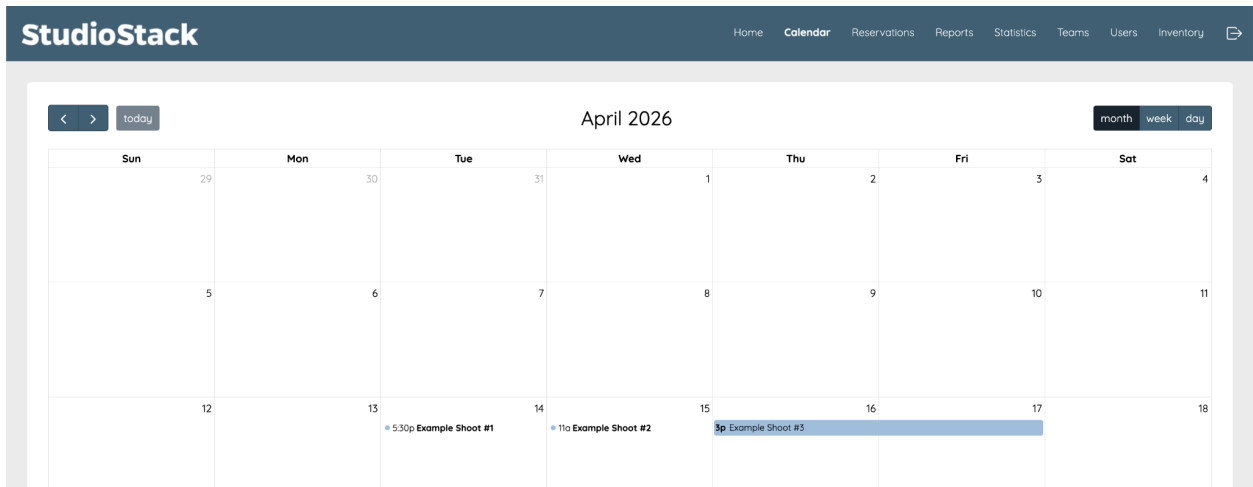
Final Presentation/Demo of how to use the application

<https://youtu.be/N3Zj-hTi4BI>

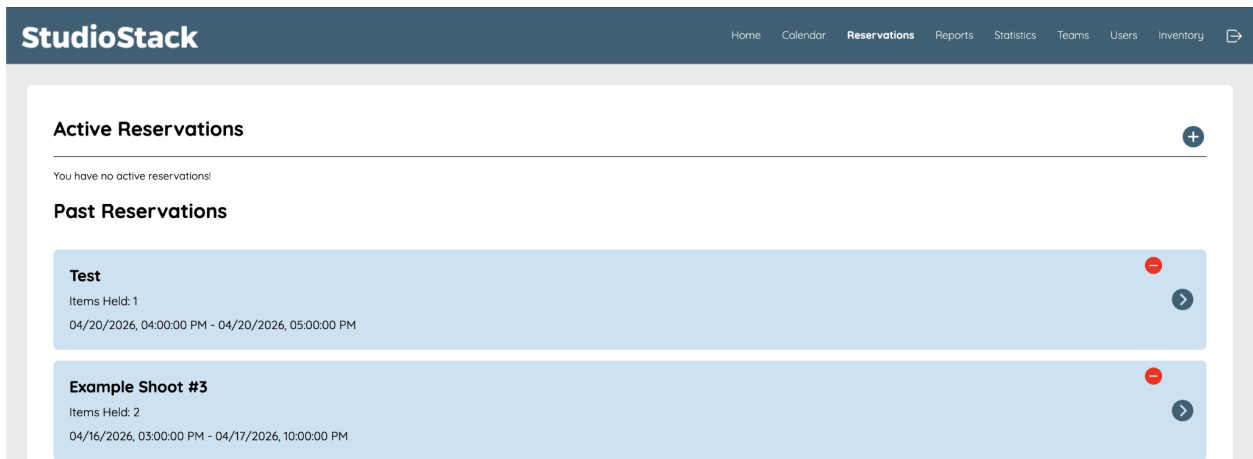
Home Page



Calendar



Reservations



Create a New Reservation

Reservation Name

ex) Bartram Photoshoot...

Choose Item(s)

Select...

Client or Internal Team

Select...

Checkout

04/21/2026

1:30 PM

Return

04/21/2026

Select time

Reserve

Reports

StudioStack

[Home](#)[Calendar](#)[Reservations](#)**Reports**[Statistics](#)[Teams](#)[Users](#)[Inventory](#)[🔗](#)

Active Reports

+

You have no active reports!

Resolved Reports

Item: Boom Mic Clip

Reported By: ash.guerra08@gmail.com

>

Item: KSIPZE LED Light Strip #1

Reported By: ztennyuk@ufl.edu

>

Report an Issue

Subject:

Select Issue...

▼

Choose Item Name:

Select Item Name...

▼

Choose Item ID:

Select Item ID...

▼

Reported By:

Select User...

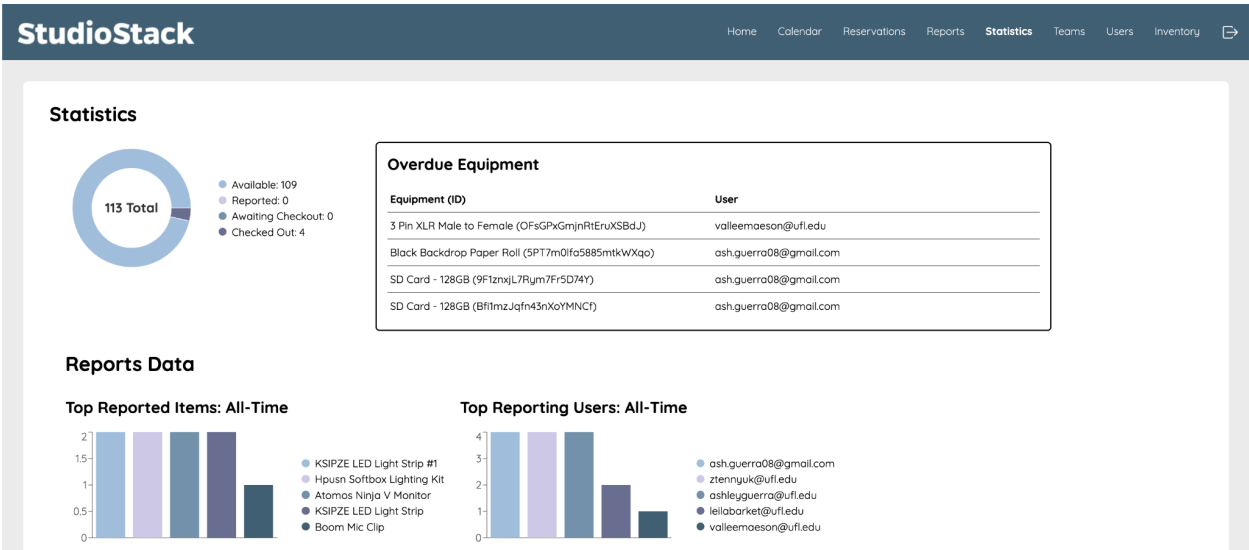
▼

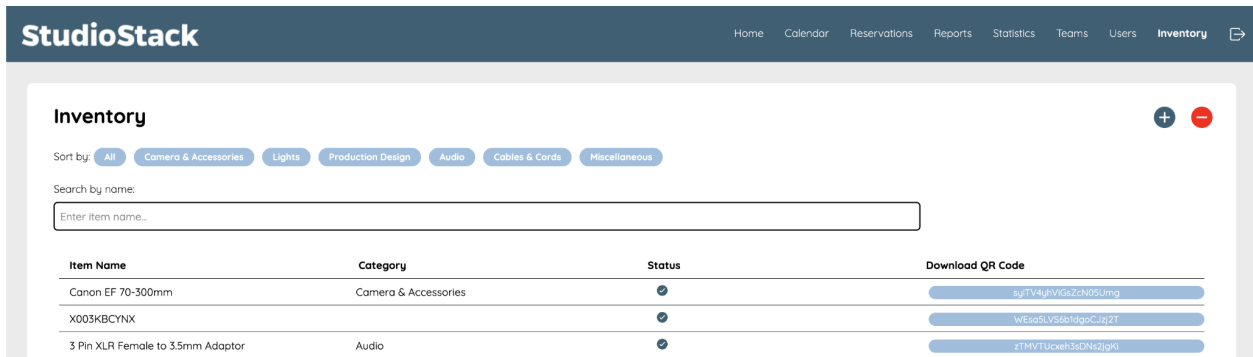
Description of the Issue:

Message...

Submit

Statistics





Bug Reporting

Report a Bug

Title

Description

Current URL

Report a Bug

Frequently Asked Questions

Why can't I log into the application?

Only approved users with UF email accounts can log in to the application. If you are unable to log in, please as an Admin user to add you to the user whitelist.

How do I check out/return items?

Users will use their device camera to scan the QR code attached to the associated piece of equipment. If a device camera isn't working, users can manually enter the associated serial code to check out/return the item.

Transition Plan/Next Steps

All materials and documentation for this project are either hosted on Agency administrative accounts or will be passed on to the Agency's Development Department leadership. The code is hosted on the Agency's GitHub. The Firebase account is linked to the Agency's Developer email. Future Agency developers will have access to all the necessary components to continue development and maintenance of the Studio Stack application.

Regarding next steps and sustainability, the application is functional and can be actively used by the Production Department. Developers can maintain active communication with the Production Department to ensure there is constant site feedback and bugs are addressed promptly.

Acknowledgments

We would like to thank our project faculty advisor, Mark Rottensteiner, for his support throughout development, as well as his feedback and suggestions for the direction of Studio Stack. Additionally, we are grateful to the Agency's Production Department, and in particular, Kyle King and Alejandra Saturno, for their feedback on how to improve the application, as well as for providing equipment to test and demo for our project showcase.

Appendices

Include brief code fragments and illustrations of results that are too verbose for the body of your report. Reference these in the body of your report. If you have a lot of raw data that you think might be interesting, you can place examples of that here as well.

Full list of tests used

- `src/pages/Report/Reports.test.jsx` > Reports > renders "no active reports" when empty
- `src/pages/Report/Reports.test.jsx` > Reports > renders active and resolved reports for admin
- `src/pages/Report/Reports.test.jsx` > Reports > only shows user reports for non-admin
- `src/components/Header.test.jsx` > Header > renders Studio Stack logo
- `src/components/Header.test.jsx` > Header > shows Admin-only links when `isAdmin` is true
- `src/components/Header.test.jsx` > Header > does not show Admin-only links when `isAdmin` is false

- `src/components/Header.test.jsx` > `Header` > shows common links for both roles
- `src/pages/LogIn/LogIn.test.jsx` > `LogIn` > renders sign in button and logo
- `src/pages/LogIn/LogIn.test.jsx` > `LogIn` > calls `setEmail` and `setIsAdmin` on successful login
- `src/pages/LogIn/LogIn.test.jsx` > `LogIn` > shows error message when user is not in database
- `src/components/ResolvedLabel.test.jsx` > `ResolvedLabel` > renders correctly with props
- `src/components/ResolvedLabel.test.jsx` > `ResolvedLabel` > has the correct background color
- `src/components/ReservationLabel.test.jsx` > `ReservationLabel` > renders correctly with props
- `src/components/ReservationLabel.test.jsx` > `ReservationLabel` > navigates to check-in-out when clicking the arrow
- `src/components/ReservationLabel.test.jsx` > `ReservationLabel` > has the correct background color
- `src/components/ReportLabel.test.jsx` > `ReportLabel` > renders correctly with props
- `src/components/ReportLabel.test.jsx` > `ReportLabel` > navigates to report-summary when clicking the arrow
- `src/components/ReportLabel.test.jsx` > `ReportLabel` > has the correct background color
- `src/pages/Reservation/Reservations.test.jsx` > `Reservations` > renders "no active reservations" when empty
- `src/pages/Reservation/Reservations.test.jsx` > `Reservations` > renders active and past reservations

Screenshot of tests running successfully

```
PS C:\Users\neema\Desktop\studiostack> npm run test

> studiostack@0.0.0 test
> vitest

DEV v4.0.18 C:/Users/neema/Desktop/studiostack

✓ src/pages/Statistics/Statistics.test.jsx (3 tests) 70ms
✓ src/pages/Calendar/Calendar.test.jsx (2 tests) 97ms
✓ src/components/ReservationLabel.test.jsx (3 tests) 90ms
✓ src/components/Header.test.jsx (4 tests) 91ms
✓ src/pages/Teams/Teams.test.jsx (3 tests) 75ms
✓ src/pages/LogIn/LogIn.test.jsx (3 tests) 142ms
✓ src/pages/Inventory/Inventory.test.jsx (3 tests) 109ms
✓ src/components/ResolvedLabel.test.jsx (2 tests) 35ms
✓ src/components/ReportLabel.test.jsx (3 tests) 42ms
✓ src/pages/Dashboard/Dashboard.test.jsx (4 tests) 57ms
✓ src/pages/Reservation/Reservations.test.jsx (2 tests) 59ms
✓ src/pages/Users/Users.test.jsx (3 tests) 57ms
✓ src/pages/Report/Reports.test.jsx (3 tests) 48ms

Test Files 13 passed (13)
Tests 38 passed (38)
Start at 16:06:56
Duration 3.53s (transform 1.13s, setup 2.66s, import 4.18s, tests 974ms, environment 11.16s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

Biography

Zachary Tenn Yuk is a fourth-year student at the University of Florida majoring in Computer Science with a minor in Economics. He has previously conducted artificial intelligence research, worked with federal government, and organized statewide educational presentations. After graduation, he will begin working as a full-time Software Engineer for Walmart Global Tech in Bentonville, Arkansas. In his free time, he enjoys cooking, going to the gym, and playing modded Minecraft.

Neema Owji is a senior studying computer science at the University of Florida with a minor in philosophy. Previously, he has worked at Siemens Energy and The Trade Desk and will be going back to The Trade Desk after graduating. His career interests are software engineering with a focus on front end development. Outside of work and school, he likes to go on hikes and try new foods.